CENTRAL CUSTOMER MANAGEMENT SYSTEM

User Story:

RYGMA SOLUTION has customers for various SaaS applications. The central Customer Management System will provide an internal platform that aggregates all customers across the various SaaS applications for easy management.

As a Rygma Admin. I want to be able to access all customers for any of our products and be able to perform some operations on those customers' accounts, like lock or suspend a customer's account, track basic activity on the customer's account and so on.

Functional Requirement:

RYGMA SOLUTION has three top products amongst others which are

1. A SaaS School Management system
2. A SaaS Computer Base Testing System that is being updated to an Enterprise version
3. A Health Management System or EMR "Electronic Medical Record" system

The scope of this Customer Management System would be limited since this is just for learning purpose, and is as follows

1. When customers Sign up on any of these products. Each product would make a request to the Customer Management System API to register the customer record into the Customer Management system
2. The Customer Management System would have a simple and intuitive User Interface that would allow a Rygma Admin to view each Product's customers
3. The Rygma Admin can perform various operations on each product customers as below
    - For the Computer Base Testing Platform, Rygma Admin should be able to activate and deactivate a Customer's account; Grant candidate slot to Customer account
    - For the School Management System, Admin should be able to activate or deactivate School Account, Upgrade School package, See the numbers of student and staff withing a School
4. The Customer Management System should expose API endpoints that can be consumed by other Rygma Products. This endpoints should be secured and can only be accessed with the right authorization

TODO:

- Each Rygma SaaS products would send different structure of data to the endpoint, therefore, investigate what database type is best suited for handling data of varying structures
- After deciding on a Database type, using docker containerization, setup the database you choose
- Create a data model that supports the Functional Requirement above
- Create backend API to support the Functional Requirement above
- Create minimalistic UI based on the designs that would be shared with you